

Implementation Plan: Java 27 Learning Curriculum

Phase: 2 (Plan) of Specify → Plan → Tasks → Implement Spec: [SPEC.md](#) Date: 2026-09-23

Verified Preconditions

Check	Result
JDK 27 installed	<code>java version "27" 2026-09-15</code> , build 27+35-2325
JDK 27 is default	Yes, <code>/Library/Java/JavaVirtualMachines/jdk-27.jdk</code>
JEP 512 compact source files work without flags	Confirmed, <code>java HelloWorld.java</code> printed <code>Hello, World!</code> , exit 0
<code>gh</code> CLI authenticated	Yes, account <code>aliturgutbozkurt</code> , <code>repo</code> scope present

The Module 01 on-ramp is proven on the real toolchain, not assumed.

Decisions Taken on Open Questions

Open questions 2 and 3 from the spec were not answered explicitly, so the spec's own recommendations stand:

- Testing (Module 21):** zero-dependency. Bare `assert` with `-ea` plus a hand-rolled micro harness, and a section explaining what JUnit adds and why real projects use it. The no-dependency rule holds.
- Preview features:** covered only in Module 22, clearly labeled, run behind `--enable-preview --source 27`. Not removed, because a learner should know the difference between stable and preview, but quarantined so nothing earlier depends on them.

Say the word if either should flip.

Components

Component	Responsibility	Depends on
scaffold	Repo skeleton, verification script, gitignore, README stub	—
template	Module 01 written as the reference implementation of the module format	scaffold
core-language	Modules 02–10, the syntax and object model a Java newcomer needs first	template
type-system	Modules 11–12, generics and collections	core-language
functional	Modules 13–15, lambdas, streams, Optional	type-system
platform	Modules 16–21, IO, stdlib, concurrency, modules, testing	functional
release-literacy	Module 22, what 26 and 27 changed, JEPs, preview vs. stable	platform
integration	Root README table of contents, full-repo verification pass	all

Build order: scaffold → template → core-language → type-system → functional → platform → release-literacy → integration

Why This Order

The ordering is pedagogical, not technical. Module authoring could in principle happen in any order once the template exists, but each module's prose assumes the learner has read the previous ones. Writing them in sequence is what keeps cross-references honest, for example Module 14 on streams referring back to the functional interfaces introduced in Module 13.

Two exceptions worth noting:

- scaffold must land first and completely. The verification script is what proves every later example runs, so writing modules before it exists means accumulating unverified content.
- template is a gate, not just the first module. Module 01 fixes the seven-section README structure, the comment style, the homework format, and the solution layout. Getting it wrong means reworking 21 modules, so it gets reviewed before Module 02 starts.

Risks and Mitigations

Risk	Impact	Mitigation
Preview features drift before finalizing	Module 22 teaches something that changes	Quarantine to Module 22, label every preview explicitly, state the JEP number and preview round so the reader can check current status
Fabricated version claims	Curriculum teaches falsehoods with confidence	Spec boundary forbids it. Every JDK claim traces to openjdk.org. Already applied: the JDK 27 JEP table came from the release page, not memory
Scope fatigue across 22 modules	Project stalls half-finished	One module per issue, each independently shippable and verifiable. A half-done curriculum of 11 verified modules still teaches
Examples silently break on a future JDK	Rot	<code>verify-examples.sh</code> is runnable in CI and covers every example and solution
Template churn after modules are written	Expensive rework	Module 01 is an explicit review gate before Module 02
Learner's JDK differs from 27	Examples fail confusingly	Root README states the required JDK, verify script checks <code>java -version</code> and fails loudly on mismatch

Parallel vs. Sequential

- **Sequential:** scaffold → template → everything else. Also Module 13 before 14, and 08 before 09, because of direct content dependencies.
- **Parallelizable:** within `platform`, Modules 16, 17, 20 and 21 have no content dependency on each other and can be written in any order or by different people at once.

Verification Checkpoints

After	Check
scaffold	<code>./scripts/verify-examples.sh</code> exits 0 on an empty tree and fails loudly when given a deliberately broken file
template	Module 01 passes verification, and its README contains all seven sections
each module	<code>./scripts/verify-examples.sh modules/<id></code> exits 0, homework brief exists, reference solution runs clean
integration	Full-repo verify exits 0, and all eight spec success criteria are checked off one by one